

Project 2

Corso di Sistemi e Architetture per Big Data

A.A. 2025/26

Valeria Cardellini, Matteo Nardelli

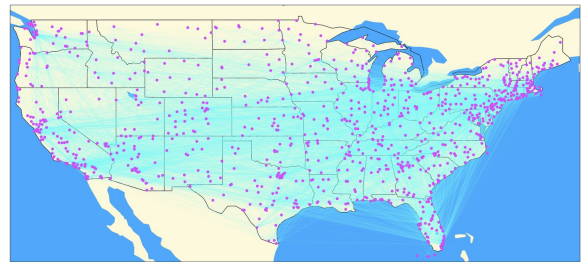
Laurea Magistrale in Ingegneria Informatica

Project delivery

- Submission deadline
 - July 7, 2026
- What to deliver
 - Link to cloud storage or repository containing project code
 - Project report composed by 4-8 pages in ACM or IEEE proceedings format
 - Presentation slides: delivered after your presentation
- Your presentation
 - July 10, 14-15, 2026 (to be confirmed)
 - Max. **15 minutes** per team
- Team
 - Target: 2 students per team
 - Alternatives: 1 student or 3 students per team
 - Send email with team composition by **June 19** only if there are changes wrt Project 1

Dataset

- Same dataset of Project 1: airline transportation
- You will analyze a dataset of **real-world airline transportation**
 - Data provided by US Department of Transportation <https://www.transtats.bts.gov/>
- Goal: analyze *in real-time* flight delays and cancellations and how different airports are impacted



Dataset

- Event = single flight characterized by:
 - Airline carrier
 - Flight date, including year, month, and day
 - Departure and arrival airports
 - Scheduled departure time
 - Departure and arrival delay, expressed in minutes; negative values indicate early flights
 - Flight status (including cancelled or diverted)
 - Causes of delay, including carrier, weather conditions, airport system, security, and previous aircraft

Project goal

- Implement a real-time analysis system of flight delays and cancellations and how they impact different airports
- Flight events are submitted as a **continuous stream** (not a batch file!) simulating the real-time flow of information, with possibly **out-of-order events**

Data streaming requirements

- Replay dataset as a stream of events
- Stream granularity: 1 event = 1 flight
 - Each record in the dataset is processed as an individual flight event
- Flight events are ordered by event time (flight date + scheduled departure time)
- Emit events ordered by:
 - Flight date
 - Scheduled departure time
- Compute event time from:
 - YEAR, MONTH, DAY_OF_MONTH, CRS_DEP_TIME
- Handle out-of-order events with:
 - Watermarking strategy
 - Maximum allowed lateness

Data streaming requirements

- Ingestion options:
 - Kafka, Pulsar, NiFi, or file stream
- Define handling of missing numeric values:
 - For example, null or 0 (justify choice)
- Delay statistics:
 - Include only non-cancelled and non-diverted flights

Queries with Flink

- Use [Flink](#) framework to answer some queries on the dataset
 - Programming language? Your choice
- Include in your report/slides queries' [latency time and throughput](#) on your reference platform

Queries with Flink

Query 1

- Goal: airline operational monitoring
- Airlines: AA, DL, UA, WN
- Aggregate events using 1-hour tumbling windows based on event time
- Compute metrics per airline and window
 - Total flights
 - Completed flights (not cancelled and not diverted)
 - Cancelled flights
 - Diverted flights
 - Average departure delay (DEP_DELAY)
 - Cancellation rate (%)
 - Departure delay rate (%) with DEP_DELAY > 15 min

Queries with Flink

Query 1

- Output schema
window start, window end, airline, num flights,
completed, cancelled, diverted, dep delay mean,
cancellation rate, late departure rate

Example:

```
# window_start, window_end, airline, num_flights, completed, cancelled, diverted,  
# dep_delay_mean, cancellation_rate, late_departure_rate  
2025-01-01 08:00:00, 2025-01-01 09:00:00, AA, 124, 118, 3, 3, 11.42, 2.42, 23.73  
2025-01-01 08:00:00, 2025-01-01 09:00:00, DL, 117, 113, 1, 3, 8.95, 0.85, 18.58
```

Queries with Flink

Query 2

- Goal: airports with the most significant delays
- Aggregate events by departure airport (ORIGIN_AIRPORT_ID) using event-time windows
- Windows for event-time processing
 - 1 hour, 6 hours, since start of dataset
- Significant departure delay:
 - Flight not cancelled
 - Flight not diverted
 - DEP_DELAY > 30 min
- Output: top-10 airports
 - Ranked by number of significantly delayed departures

Queries with Flink

Query 2

- Metrics per airport
 - Total flights
 - Significant delays count
 - Average DEP_DELAY
 - Maximum DEP_DELAY
 - Top 20 delayed flights (airline, destination airport, DEP_DELAY)
- Output schema
ts, rank, origin airport id, num flights, severe delays, dep delay mean, dep delay max, delayed flights

Example:

```
# ts, rank, origin_airport_id, num_flights, severe_delays,  
# dep_delay_mean, dep_delay_max, delayed_flights  
  
2025-01-03 12:00:00, 1, 10397, 298, 17, 18.42, 214.00,  
[(AA,11298,214.00), (DL,13930,176.00), ...]  
2025-01-03 12:00:00, 2, 11292, 271, 14, 16.87, 188.00,  
[(UA,12892,188.00), (WN,14107,165.00), ...]
```

Queries with Flink

Query 3

- Goal: real-time departure delay distribution across airlines and times of day
- Consider flights operated by:
 - AA, DL, UA, WN
- Only non-cancelled and non-diverted flights
- Group by:
 - Airline
 - Scheduled departure hour (24 hourly slots)
- Windows for event-time processing
 - 1 day, 7 days, since start of dataset (global window)

Queries with Flink

Query 3

- For each airline and each time slot, compute:
 - Number of flights
 - Minimum DEP_DELAY
 - 25th percentile
 - 50th percentile (median)
 - 75th percentile
 - 90th percentile
 - Maximum DEP_DELAY
- Requirements
 - Real-time percentile estimation
 - Use an approximate quantile algorithm
 - Avoid storing/sorting all observations
 - Describe algorithm in the report

Queries with Flink

Query 3

- Output schema

ts, airline, hour, count, min, p25, p50, p75, p90, max

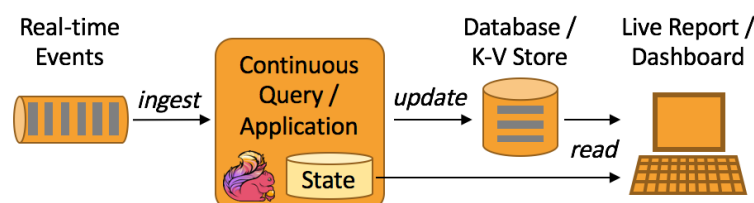
Example:

```
# ts, airline, hour, count, min, p25, p50, p75, p90, max
2025-01-01 00:00:00, AA, 8, 421, -12.00, -3.00, 4.00, 18.00, 47.00, 214.00
2025-01-01 00:00:00, DL, 8, 398, -15.00, -4.00, 3.00, 15.00, 39.00, 176.00
...
```

Data ingestion

- We are now focusing on streaming analytics
 - Rather than processing the entire dataset in batch mode, we **ingest real-time event streams**, enabling Flink to continuously analyze data and update results as events arrive
- Replay the stream event by event
- Preserve the relative timing between events, while applying an appropriate time-scaling factor

Streaming analytics



Platform and performance evaluation

- Evaluate the application experimentally on your reference platform by measuring:
 - **Latency**: time required to process an event
 - **Throughput**: number of events processed per time unit
- Your platform can be a standalone node
 - Recommended: use Docker Compose to orchestrate the containers running on the same machine
- Alternatively, you can use a cloud service for big data processing (e.g., Amazon Managed Service for Apache Flink) with the AWS Academy grant

Team of 1 or 2 students

- 2 students:
 - Address Q1, Q2, and Q3
 - Evaluate performance metrics
- 1 student:
 - Address Q1 and Q2
 - Evaluate performance metrics
 - Optional: full ingestion pipeline
 - Mandatory: simulate the input stream (event replay), process data as a stream, export query results to CSV files

Team of 3 students

- Address Q1, Q2, and Q3
- Evaluate performance metrics
- Consider the following alternatives:
 - A. Watermarking evaluation
 - Detect out-of-order events
 - Compare ≥ 2 watermark strategies
 - Evaluate: result completeness and processing latency
 - B. Framework comparison
 - Implement all queries also in Spark Streaming
 - Evaluate and compare their performance on the same reference platform
 - Analyze key metrics such as latency, throughput, and resource utilization

Optional

- Generate a Grafana dashboard (or an equivalent visualization tool) to display the temporal evolution of the query results
 - You may also include performance evaluation metrics